

문제해결과 알고리즘

러닝페어#4

[최종보고서]

팀 이름	18미만A+금지
구성원 이름 학번	조성원(2018312863) 황준영(2018312981) 문성재(2018310806) 이원경(2018311884)

목 차

1. 개요	1
1.1 프로젝트 개요	1
1.2 주제 선정 동기	1
1.3 프로젝트 목표	1
2. 개발 내용 분석 및 설계.....	2
2.1 기존 프로그램과의 차별성.....	2
2.2 프로그램 전체 시나리오	3
2.3 기능 세부 설계	3
3. 최종 결과.....	3
3.1 프로젝트 결과	4
3.2 문제 해결 방법	8
3.3 팀원 업무 분배	10
4. 참고 문헌.....	10

1. 개요

1.1 프로젝트 개요

한 시장조사에 의하면, 성인남녀가 결정장애를 가장 절감하는 경우는 '음식메뉴를 고를 때'로 나타났다. 현대를 살아가며 사람들은 연속적인 선택을 맞닥뜨리며, 작은 문제에도 결정의 어려움을 크게 느낀다. 이 프로젝트는 이러한 사람들의 니즈를 해결하기 위해 시작했다. 넋지 행동경제학에 의하면 결정장애의 원인은 선택지가 지나치게 많기 때문이다. 따라서, '선택설계'를 통해 사람들의 선택에 적당히 개입하여 선택지를 줄이되, 선택의 자유는 개인에게 맡겨야만 사람들의 이용률을 높힐 수 있다. 바쁜 업무에 지친 직장인, 과제가 많은 대학생, 외식 메뉴를 정하기 힘든 가족이나 동호회 단체들이 시간을 절약할 수 있을 것이다

1.2 주제 선정 동기

두잇 서베이 시장조사에 의하면, 성인남녀가 결정장애를 가장 절감하는 경우는 '음식메뉴를 고를 때'로 나타났다. 실제로 음식 결정장애를 위한 메뉴판이 인터넷 밈으로 유행하기도 했다. 랜덤으로 음식 메뉴를 지정해주는 어플리케이션이 등장하기도 했지만, 사람들의 선호를 전혀 반영하지 못해 큰 인기를 끌지 못했다. 따라서, 음식 메뉴를 서서히 좁혀나가는 선택지를 제시하고, 그 과정에서 사람들의 최근 식사 패턴이 적절히 반영될 수 있도록 서비스를 설계하고자 했다. 또한, 기존 유저들은 자신이 방문한 맛집만 편리하게 기록할 수 있는 공간이 없어 블로그나 스마트폰 메모앱을 이용해야 때문에, 이러한 불편함을 해결하고자 맛집 만을 편리하게 기록하는 캘린더 기능을 기획에 추가했다

1.3 프로젝트 목표

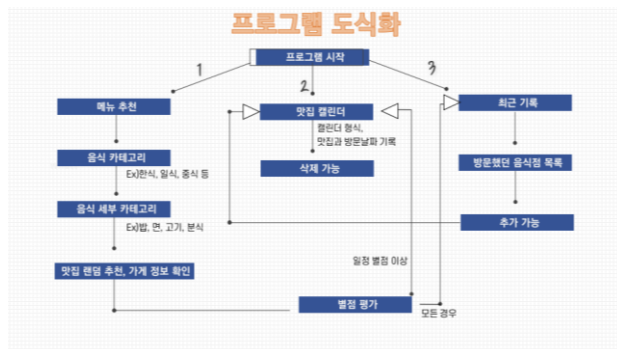
현대 사회에서 사람들은 살면서 많은 선택지를 마주하고, 선택 과부하는 의사결정 과정에 좌절감을 주고 결과에 불만족하도록 한다. 그러나, 사소해 보이는 음식 식사메뉴 결정은 실제로 삶의 만족도와 깊게 연관되어 있다. 계층별로 먹거리와 삶의 질의 상관관 계에 대해 분석한 결과, '전반적인 삶의 행복도'와 '먹거리·식생활 만족도'의 관련성을 보여주는 상관성 계수(668)이 매우 높은 수준으로 나타났다. 따라서, 사람들의 음식 메뉴 선정 알고리즘을 통해 사람들의 적절한 음식 메뉴 선정을 돕는다면, 전반적인 삶의 만족도에도 기여할 수 있을 것이다.

2. 개발 내용 분석

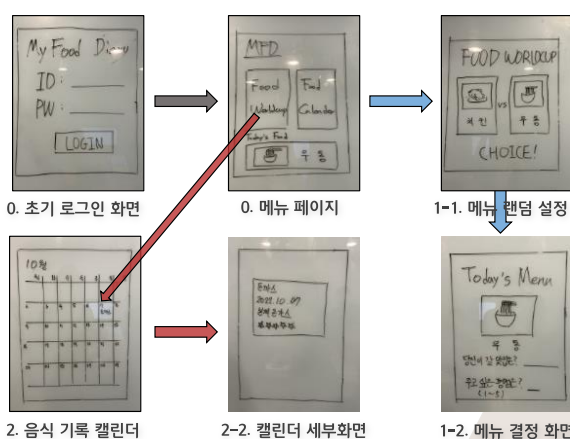
2.1 기존 프로그램과의 차별성

기존 메뉴 선택 프로그램의 경우 선택지가 충분히 다양하지 않거나, 메뉴가 랜덤으로 추천되어 사람들이 결과에 만족하지 않는 경우가 많았다('오늘 뭐먹지?' 어플리케이션). 이에 기존 개괄적인 음식 메뉴 제시에 그치는 것이 아닌 보다 다양한 선택지를 제공하고자 했다. 또한, 사람들이 자신의 선호도를 바탕으로 선택지를 좁혀나가는 과정에서, 최근 먹은 메뉴를 팝업 형식으로 띄워 이용자들이 만족하는 결과를 도출할 수 있도록 했다. 기존 네이버와 카카오맵, 배달의 민족의 경우 광고나 리뷰이벤트로 부풀려져 이용자들이 원하는 맛집을 찾기가 쉽지 않았다. 따라서 사람들은 자신의 경험을 바탕으로 맛집을 기록해 해당 맛집을 재방문하는 경우가 많았다. 그러나 맛집만을 편리하게 관리할 수 있는 프로그램이 없어 메모장, 어플이나 네이버 블로그 등의 귀찮은 절차를 거쳐야 했다. 이러한 불편함을 캐치하여 나만의 맛집 캘린더 기능을 추가하였다.

2.2 프로그램 전체 시나리오



2.3 기능 세부 설계

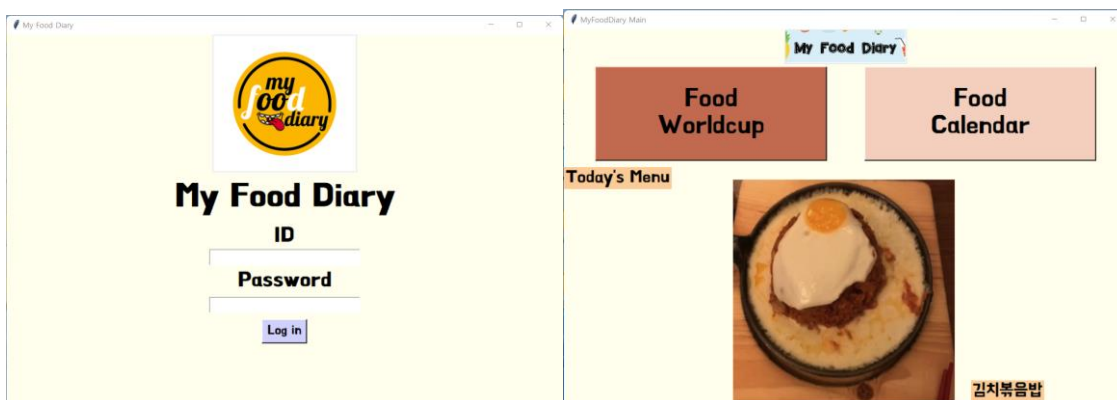


로그인 후 메뉴 페이지 좌측 상단에 있는 Food worldcup을 선택하면 두 가지 메뉴 중 하나를 선택하게 된다. 토너먼트 형식으로 구성된 단계들을 거치면 최종적으로 음식 추천이 완료된다(1-2). 사용자가 이 메뉴에 만족한다면 방문한 식당과 해당 식당의 별점을 함께 기록하여 이를 캘린더에 저장한다. 다시 초기 메뉴로 돌아가 음식 캘린더를 들어가면, 날짜별로 그날 먹은 메뉴, 방문한 식당, 이용자가 기록한 별점을 확인할 수 있다(2-2).

3. 최종 결과

3.1 프로젝트 결과

[실행 결과 화면] ①



[이에 해당하는 코드]

```

path = './data' #음식사진 파일들의 list를 만들고 random.shuffle을 이용해 list를 무작위로 섞어줌
lst = os.listdir(path)
random.shuffle(lst)
global store_user_name

def rand_food(win):
    global img_r
    img_r = PhotoImage(file = f'./data/{lst[0]}', master=win) #무작위로 섞은 음식 사진파일 list중 첫번째 element를 가져와 랜덤 추천 메뉴로 띄워줌
    img_r = img_r.subsample(9)
    #img_r =
    lab_r = Label(win, width=350, height=950)
    lab_r.config(image = img_r)
    lab_r.place(x=270, y=240)

    foodName = lst[0].replace('.png', '') #추천 메뉴로 띄운 사진파일 이름에서 '.png'를 뺀 음식이름을 출력해줌
    lab_n = Label(win, text = foodName, font=('Arial', 20), fg = 'hotpink', bg = 'white')
    lab_n.place(x=650, y=560)
    #lab_r.pack()

def start_main_view(user_name): #MyFoodDiary의 메인 화면을 보여줌
    global store_user_name
    store_user_name = user_name
    win = Tk()
    win.title("MyFoodDiary Main")
    win.geometry("300x500+500+500")
    win.option_add("-font", "Arial 20")
    win.config(bg='ivory')

    lab_l = Label(win)
    img = PhotoImage(file = './photo/MFD.png', master=win) #MyFoodDiary title 사진을 메인 화면 위쪽에 출력해줌
    img = img.subsample(2)
    lab_l.config(image = img)
    lab_l.pack()

    def launch_food_world_cup(): #FoodWorldcup 실행 코드
        win.destroy()
        FoodWorldCup.load_food_world_cup(store_user_name)
        start_main_view(store_user_name)

    def launch_calendar(): #FoodCalendar 실행코드
        showCalendar.openWeb(calendar_address)

    bt1 = Button(win, text="FoodWorldCup", command=launch_food_world_cup) #FoodWorldcup 버튼 클릭시 FoodWorldcup을 실행시켜줌
    bt1.config(bg='skyblue')

    bt2 = Button(win, text="FoodWnCalendar", command=launch_calendar) #FoodCalendar 버튼 클릭시 FoodCalendar를 실행시켜줌
    bt2.config(bg='skyblue')

    bt1.place(x=50, y=50, width=370, height=150)
    bt2.place(x=480, y=50, width=370, height=150)

    lab_fr = Label(text = "Today's Menu", font=('Arial', 20), bg = 'yellowgreen') #오늘의 추천 메뉴 화면
    lab_fr.place(x=0, y=220)
    rand_food(win) #rand_food함수를 호출해 랜덤 음식 사진파일과 이름을 화면에 출력해줌
    win.mainloop()

if __name__ == "__main__":
    start_main_view("hello")

```

[코드 설명] : 음식사진 파일들이 있는 폴더를 path로 가져와서 음식사진 파일들의 list를 만들어주고 random.shuffle을 이용해 list를 무작위로 섞어준다. 그 후 rand_food함수를 정의해서 무작위로 섞은 음식사진 파일 list에서 첫번째 element의 사진과 파일명을 랜덤 추천 메뉴 사진과 이름으로 띄워준다. 그 후 MyFoodDiary의 메인 화면을 구성해서 보여준다. 이때 MyFoodDiary title 사진을 불러와서 메인 화면의 위쪽에 출력해준다. 그리고 FoodWorldcup을 실행해주는 launch_food_world_cup함수와 FoodCalendar를 실행해주는 launch_calendar함수를 정의해준다. 그 후 각각의 버튼을 정의해서 화면에 띄워주고 버튼에 command를 줘서 각 버튼을 클릭 시 FoodWorldcup과 FoodCalendar를 실행시키도록 한다. 마지막으로 오늘의 추천 메뉴 화면은 rand_food함수를 호출해서 추천 메뉴 사진과 이름을 화면에 출력해준다.

[실행 결과 화면] ②



Notion DB를 이용한 로그인 기능

```
def checkIdandPW(ID: str, PW:str, database_id, token):
    """특정 ID와, PW로 user가 존재하는지 체크하는 Function

    Args:
        ID (str): user가 입력한 ID
        PW (str): user가 입력한 PW
    """

    specific_user = queryUser(database_id, token, "ID", "title", ID)
    if not specific_user:
        return makeReponseDict("Failed", "No ID in Database")
    user_info = specific_user[0]
    user_pw_in_database = user_info["PW"]
    if(user_pw_in_database != PW):
        return makeReponseDict("Failed", "Wrong Password")
    return makeReponseDict("Success", user_info=user_info)
```

```
def jsonToUserList(user_json): #Post로 받은 유저 데이터 파싱
    user_list = list()
    for user in user_json['results']:
        try:
            user_dict = dict()
            properties = user['properties']
            id = properties['ID']['title'][0]['text']['content']
            pw = properties['PW']['rich_text'][0]['text']['content']
            name = properties['이름']['rich_text'][0]['text']['content']
            age = properties['나이']['rich_text'][0]['text']['content']
            user_dict["ID"] = id; user_dict["PW"] = pw; user_dict["NAME"] = name; user_dict["AGE"] = age
            user_list.append(user_dict)
        except:
            continue
    return user_list

def postToNotion(url, payload, headers): #Notion에 Post
    response = requests.post(url, json=payload, headers=headers)
    user_json = response.json()
    return jsonToUserList(user_json)
```

```
def makeReponseDict(request_text="", reason_text="", user_info: dict = ""):
    """response 객체를 생성하는 Function

    Args:
        request_text (str, optional): 쿼리 요청 성공 or 실패 여부. Defaults to "".
        reason_text (str, optional): 요청이 실패한 경우, 실패 이유. Defaults to "".
        user_info (dict) : 요청이 성공한 경우, 유저의 정보를 전달
    """
    return {"Request": f"{request_text}", "Reason": f"{reason_text}", "user_info" : user_info}
```

[코드 설명] : User가 입력한 ID, PW, 쿼리를 요청하는 **database ID, token**을 입력을 받아서, Notion API에 request를 보냅니다. HTTP response (JSON)으로 받은 user info 내용을 파싱하여, **user가 존재하지 않는 경우, Password가 일치하지 않는 경우, ID, PW를 맞게 입력한 경우를 구분** 하였습니다. 최종적으로, 위 내용을 바탕으로 Response에 대한 dictionary 객체를 return 하게 됩니다.

[실행결과 화면]③



[이에 해당하는 코드] 각 라운드마다 이미지를 저장할 공간과 초기화 함수

```

# Execute Food World Cup
class FoodWorldCup:
    # Space to store values for each round
    c0 = []
    c1 = []
    c2 = []
    c4 = []
    c8 = []
    c16 = []
    c32 = []
    food_db = []
    f_cup = {}
    fname = [] # Food name
    rounds_num = {3: 32}

    list = os.listdir('./data') # Path to importing pictures of food

    # Get the picture file name and mix it randomly
    def setting_db(self):
        for names in self.list:
            self.fname.append(names) # Adding Food Photos
            random.shuffle(self.fname) # Randomly shuffle

    # Initialize Values
    def setting_value(self):
        self.c1 = []
        self.c2.clear()
        self.c4.clear()
        self.c8.clear()
        self.c16.clear()
        self.c32.clear()
        self.c0.clear()
        self.food_db.clear()
        self.fname.clear()
        self.f_cup = {1: self.c1, 2: self.c2, 4: self.c4, 8: self.c8, 16: self.c16, 32: self.c32}
  
```

[코드 설명] : 32강부터 결승전까지 빈 리스트를 생성해주고 setting_db 함수를 통해 리스트 안에 들어가 있는 음식 이미지들을 랜덤으로 섞어 넣어준다. setting_value에서는 이미 한 번 실행된 후

저장되어 있는 음식 이미지들을 다시 초기화를 하여 다시 실행했을 때, 사용자가 원하는 새로운 결과 값이 나오도록 해준다.

[Food WorldCup에서 사용자가 선택하는 음식을 상위 라운드로 가져오는 함수]

```
# Running WorldCup
def select_items(self, round):
    get_name = lambda x: x.split('.')[0] # Only file names
    random.shuffle(self.f_cup[round]) # Mix the values of the list randomly
    for i in range(0, round, 2):

        def btnpress(): # Functions executed by pressing the left button

            original_db_number = self.f_cup[round][i] # User selects left picture

            if round == 2: # In final
                self.final_view(round, original_db_number)
            else: # Not final
                # Append the results selected in the current round to the list in the next round
                self.f_cup[int(round / 2)].append(original_db_number)

            w.destroy()

        def btnpress2(): # Functions executed by pressing the right button

            original_db_number = self.f_cup[round][i + 1] # User selects right picture

            if round == 2: # Final
                self.final_view(round, original_db_number)
            else: # Not Final
                # Append the results selected in the current round to the list in the next round
                self.f_cup[int(round / 2)].append(original_db_number)

            w.destroy()
```

[코드 설명]

이 부분은 Food WorldCup에서 사용자가 고르는 음식 사진을 상위 라운드로 이동시키는 함수이다. get_name 변수를 통해 음식의 이름만 가져온 후, 랜덤으로 섞어 라운드가 시작되도록 한다. 32강부터 결승전까지 for문을 통해 반복된다. btnpress() 함수는 사용자가 왼쪽 사진을 클릭했을 때, btnpress2() 함수는 사용자가 오른쪽 사진을 선택했을 때 동작하는 함수로 결승전이 아닐 경우에는 그 선택된 함수가 다음 라운드의 리스트에 추가될 수 있도록 append하고, 결승전일 경우에는 그 최종 선택된 음식 이미지를 가지고 final_view 함수를 실행시킨다.

[Food WorldCup의 첫 화면 구성]

```
# Food WorldCup display
w = Tk()
w.geometry("800x800+500+500")
w.title("Food WorldCup")
w.config(bg="ivory")

# Text
label = Label(w, text="Food World Cup", font=('Arial', 30), bg="ivory", fg="black")
label.place(x=310, y=20)
label = Label(w, text="({})".format(round), font=('Arial', 15), bg="ivory", fg="black")
label.place(x=430, y=280)

# Left Picture
food1 = "/data/" + self.fname[self.f_cup[round][1]]

photo = PhotoImage(file=food1)
photo = photo.subsample(10)
btn = Button(w, image=photo, text=get_name(self.fname[self.f_cup[round][1]]), command="top", bg="ivory")
btn.config(x1=400, y1=400, height=400, font=('Arial', 30), )
btn.pack(side="left")
btn.config(command=btnpress)

# Right Picture
food2 = "/data/" + self.fname[self.f_cup[round][1 + 1]]

photo2 = PhotoImage(file=food2)
photo2 = photo2.subsample(10)
btn2 = Button(w, image=photo2, text=get_name(self.fname[self.f_cup[round][1 + 1]]), font=('Arial', 30),
              command="top", bg="ivory")
btn2.config(x1=400, y1=400, height=400)
btn2.pack(side="right")
btn2.config(command=btnpress2)

w.mainloop()

if round != 2:
    round = int(round / 2)
    self.select_items(round)
```

[코드 설명] : Tkinter을 이용하여 view를 띄웠다. Text는 label을 사용하였고, 음식 이미지의 경우 이미지 버튼을 사용하여 구성하였다. 왼쪽 이미지를 food1, 오른쪽 이미지를 food2로 설정하여 path를 통해 불러오고, 이를 이미지 버튼의 이미지로 설정하여 음식 사진을 클릭할 수 있도록 코딩하였다. 버튼을 누를 경우, 어떤 버튼을 클릭했느냐에 따라 위에서 말한 함수 2개 중 하나가 실행되고 사용자가 선택한 음식이 상위 라운드로 이동하게 되는 것이다.

[Food WorldCup이 실행될 때, 작동하는 함수]

```
# Round setting
def setting(self): # Initialization
    self.setting_value() # Initialize values
    self.setting_db() # Initialize Food_DB
    rounds = self.rounds_num[3]
    for round in range(rounds):
        self.c0.append(round)
    self.f_cup[rounds] = self.c0 # Initialized values
    self.select_items(rounds)

# Start
def start(self):
    self.setting()
```

[코드 설명]

setting() 함수를 통해 사용자가 원하는 음식 사진을 다시 고를 수 있도록 한번 진행되어 저장되어 있던 Food WorldCup의 데이터들을 다시 초기화한다. 메인 함수에서 FoodWorldcup().start()를 실행한다면 start 함수가 setting()을 실행하여 새로운 Food WorldCup이 진행되도록 한다.

3.2 문제 해결 방법

①2주차에서 학습한 문제 해결과정, 수렴적 사고 개념을 통한 해결책 도출

사람들에게 도움이 되는 프로그램이 되려면 무엇보다 적절한 해결책을 제시하는 것이 중요하다. 가장 최선의 해결책을 선택하기 위해서는 수렴적 사고가 필요하다. 이는, 더 유용하고 효율적이며 가치 있는 것을 선택하기 위한 사고이다. '식사 메뉴 결정의 어려움'이라는 문제를 해결하기 위해 룰렛 형식의 랜덤 메뉴 추천, 어제 먹은 메뉴를 배제하는 추천 알고리즘, 음식 토너먼트 형식 등 다양한 해결책이 제시되었다. 이때, 첫 번째로 떠오른 해결책에 만족하지 않고, 보다 더 효율적이면서도 사용자들을 만족시킬 수 있는 해결책을 찾으려 했다. 그 결과, 양자택일 형식으로 선택은 빠르고 쉽게 이루어지되, 토너먼트 형식으로 사람들의 선호가 충분히 반영될 수 있는 푸드 월드컵을 도출할 수 있었다. 또한, 고민 끝에 캘린더 기능을 추가하였고, 이는 사용자가 기록된 음식 메뉴를 바탕으로 선택을 좀 더 쉽게 할 수 있게끔 유도한 것이다.

②큐를 활용한 토너먼트 구성

큐 자료구조는 먼저 넣은 데이터가 먼저 나오는 FIFO 구조로 저장하는 형식을 의미한다. 수업시간에 활용한 큐 자료구조를 활용하여 토너먼트를 구성하였다. 먼저, 랜덤으로 섞은 32개의 숫자를 큐에 삽입하였다. 이후, 큐에서 두 개씩 숫자를 추출하고, 선택된 음식을 다시 큐에 삽입하고 이 과정을 마지막 1개가 남을 때까지 진행하였다.

③GUI 활용

CLI는 숫자 또는 문자들을 키보드를 통해 입력 받고, 그 결과를 문자의 형태로 모니터에 표시하는 방식이다. 이와 달리, GUI는 화면의 특정 지점을 클릭, 드래그 하는 등의 방법으로 사용자의 명령을 컴퓨터에게 전달한다.

GUI는 사용자가 직관적으로 조작 방법을 이해할 수 있게 고안되어 있는 것이 장점이다. 'My food diary'는 사용자들의 불편을 최소화하기 위해 고안되었기 때문에, GUI를 사용하여 구현하였고, tkinter모듈을 사용하였다.

④Notion API 활용

프로젝트 협업 툴 중의 하나인 Notion은 **User defined Database**를 제공하고 있다. 상용 데이터베이스 (AWS RDS, Microsoft Azure 등)는 API를 사용할 경우 사용량당 과금을 하는 방식을 취하고 있다. 그러나 Notion API의 경우, **과금 없이 Database Query**문을 작성해서 HTTP request(POST, GET)를 보냄으로써, 데이터베이스를 삭제, 추가, 변경이 가능하기에 데이터베이스 전체를 관리할 필요 없이 Food diary를 이용하는 유저의 정보(ID, PW, 이름, 나이) 등을 가져올 수 있다.

또한, Notion Database는 **다양한 View 방식**(표, 캘린더 등)을 제공하므로 저장한 정보를 원하는 view에 맞게 띄워주면 되는 편리함 또는 갖추고 있다.

위의 기능을 이용하여, **Food diary의 Login 기능, Food Calendar**의 기능을 구현하였다.

3.3 팀원 업무 분배

이름	학번	학과 (계열)	주제 선정 단계	설계 단계	구현 단계	최종 확인 단계
			09.13 ~ 09.22	10.7 ~ 10.23	10.24 ~ 11.13	11.13 ~ 11.20
문성재	2018310806	소프트웨어학과	-자료조사 및 개발 계획서 검토 -주제 3개 제시 -개발계획서의 '프로젝트 개요'작성	- 프로그램 시나리오 및 화면 작성(로그인 화면) - 중간 보고서 참고 문헌 정리 -Notion Database API 개발 문서 확인 및 API 테스트	-Notion API를 이용한 Notion DB 유저 정보 쿼리 및 유저 정보 추가 python파일 작성	-최종 발표자료 제작 -시연 영상 녹화
이원경	2018311884	프랑스어문학과 (인문과학계열)	- 자료조사 및 개발계획서 검토 - 주제 3개 제시 - 개발계획서의 '프로젝트 주제 선정 동기' 작성	-프로그램 시나리오 및 화면 작성 (주요 메뉴 화면) -(Tkinter). Python GUI 함수 이해	-주요 메뉴 화면 디자인 및 GUI로 구현 -Food World Cup에 들어갈 사진 수집	-최종 발표영상 녹화 -최종 발표자료 제작
조성원	2018312863	소프트웨어학과	- 주제 3개 제시 - 개발계획서의 '프로젝트 목표' 작성	-프로그램 시나리오 및 화면 작성 (Food worldcup) - (Tkinter). Python GUI 함수 이해	-random을 이용한 오늘의 음식메뉴 선정기능구현 - Tkinter를 이용한 Food World Cup 구현	-최종보고서 3.1 프로젝트 결과 작성
황준영	2018312981	소프트웨어학과	- 주제 3개 제시 - 개발계획서 '기존 프로그램과의 차별성' 작성	-프로그램 시나리오 및 화면 작성 (음식 캘린더) -Notion calendar API pywebview 이해 및 테스트	-Notion API를 이용해 서캘린더에 음식정보 추가하는 파일 작성 -Pywebview를 이용하여 음식 캘린더 구현	-최종보고서 3.2 문제해결 방법 작성

4. 참고 문헌

[1] "결정장애 절감하는 경우 1위 '음식 고를 때', 결정의 어려움 겪는 이유?", 권연수 기자, 2018.01.22

http://digitalchosun.dizzo.com/site/data/html_dir/2018/01/22/2018012210787.html

[2] "선택의 연속 '네티지'의 시대"를 사는 지혜", 김종면 교수, 2017.11.1

<https://www.korea.kr/news/policyBriefingView.do?newsId=148844259>

[3] "서울시민 '먹거리·식생활만족도' 6.95점", 이한영 기자, 우리문화신문,

2022.4.23 <https://koya-culture.com/mobile/article.html?no=135007>

[4] "Notion Developes API"

[Getting started \(notion.com\)](https://www.notion.com/getting-started)